

Результаты экспериментов с изменением положения и типа нормализации в реализации BigramLanguageModel

Владимирова Элина

27 мая 2025 г.

Содержание

Введение	1
Обучение без нормализации	2
LayerNorm	3
GroupNorm	5
Выводы	7

Введение

В рамках домашнего задания 2 были рассмотрены различные расположения и различные типы нормализации в классах Block и BigramLanguageModel. Изменения вносились в отмеченные комментариями строки кода:

```
1 class Block(nn.Module):
2     """ Transformer block: communication followed by computation """
3     def __init__(self, n_embd, n_head):
4         <...>
5         self.ln1 = nn.LayerNorm(n_embd) # here normalization type can be changed
6         self.ln2 = nn.LayerNorm(n_embd) # here normalization type can be changed
7     def forward(self, x):
8         x = x + self.sa(self.ln1(x)) # here normalization order (or presense) can be changed
9         x = x + self.ffwd(self.ln2(x)) # here normalization order (or presense) can be changed
10        return x
11
12 class BigramLanguageModel(nn.Module):
13     """ Super simple bigram model """
14     def __init__(self):
15         <...>
16         self.ln_f = nn.LayerNorm(n_embd) # here normalization type can be changed
17         <...>
18     def forward(self, idx, targets=None):
19         <...>
20         x = self.ln_f(x) # (B,T,C) here normalization type can be changed
```

Обучение проводилось со следующими гиперпараметрами:

- `batch_size = 256`
- `block_size = 32`
- `max_iters = 5000`
- `eval_interval = 100`
- `learning_rate = 1e-3`
- `device = "cuda" if torch.cuda.is_available() else "cpu"`
- `eval_iters = 200`
- `n_embd = 64`
- `n_head = 4`
- `n_layer = 4`
- `dropout = 0.0`

Обучение без нормализации

В качестве стартового ориентира (предположительного минимума по результативности и стабильности обучения) взят код без нормализации:

```
1 def forward(self, x):
2     x = x + self.sa(x)
3     x = x + self.ffwd(x)
4     return x
```

В таблице ниже отображены результаты обучения без нормализации:

Положение нормализации	Training loss на последнем шаге	Validation loss на последнем шаге	Лучший validation loss
Без нормализации	1.5675	1.8522	1.8432

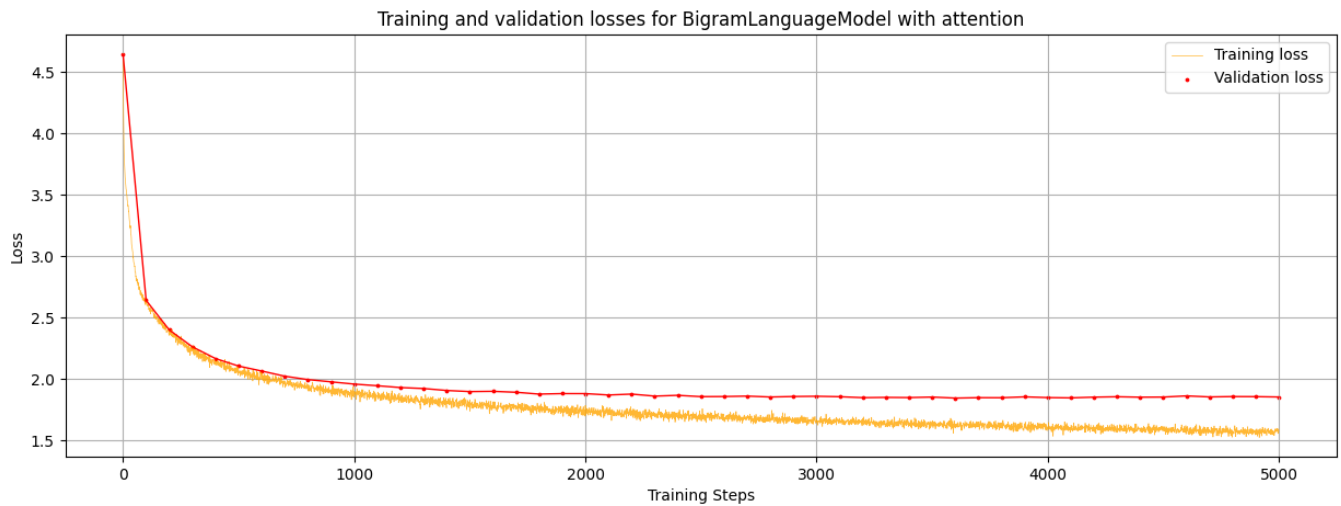


Рис. 1: Training и validation losses для BigramLanguageModel без нормализации

LayerNorm

В качестве нормализации используется стандартный `nn.LayerNorm` в 4 расположениях относительно Self-attention и Feed-forward:

1. Нормализация аргументов Self-attention и Feed-forward (Pre-LN):

```
1 def forward(self, x):
2     x = x + self.sa(self.ln1(x))
3     x = x + self.ffwd(self.ln2(x))
4     return x
```

2. нормализация результатов Self-attention и Feed-forward (Post-LN):

```
1 def forward(self, x):
2     x = x + self.ln1(self.sa(x))
3     x = x + self.ln2(self.ffwd(x))
4     return x
```

3. нормализация аргументов и результатов Self-attention и Feed-forward (Sandwich-LN):

```
1 def forward(self, x):
2     x = x + self.ln1(self.sa(self.ln1(x)))
3     x = x + self.ln2(self.ffwd(self.ln2(x)))
4     return x
```

4. нормализация аргумента Self-attention и результата Feed-forward (Pre-Post-LN):

```
1 def forward(self, x):
2     x = x + self.sa(self.ln1(x))
3     x = x + self.ln1(self.ffwd(x))
4     return x
```

В таблице ниже отображены результаты сессий обучения с различными расположениями послойной нормализации¹. В каждом столбце жирным шрифтом выделен лучший показатель.

Положение нормализации	Training loss на последнем шаге	Validation loss на последнем шаге	Лучший validation loss
Pre-LN	1.5294	1.8521	1.8308
Post-LN	1.3426	1.9036	1.8098
Sandwich-LN	1.4869	1.8835	1.8450
Pre-Post-LN	1.4899	1.8666	1.8314

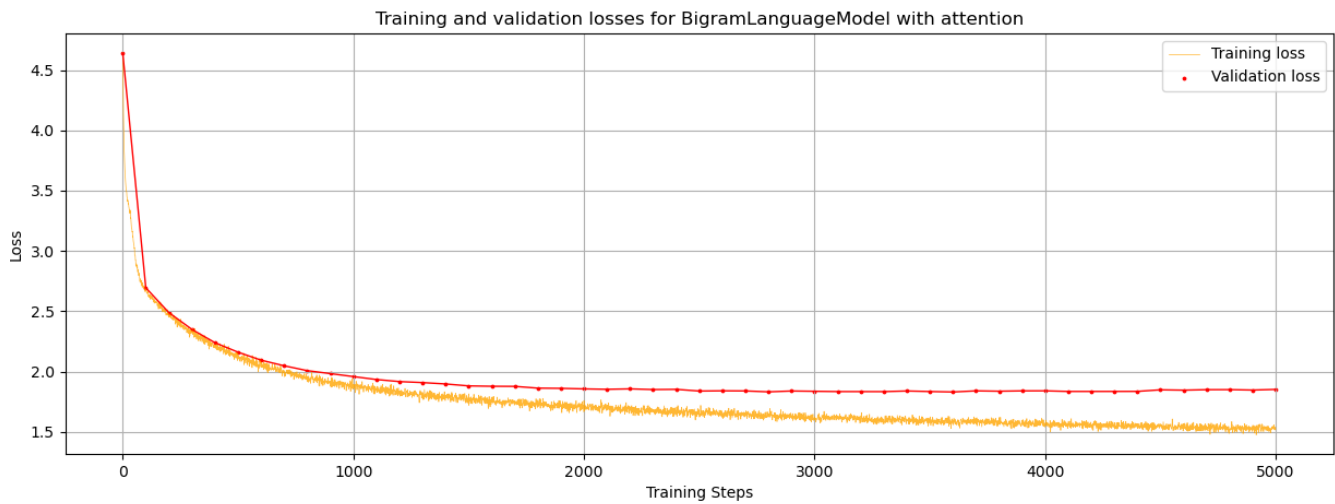


Рис. 2: Training и validation losses для BigramLanguageModel с нормализацией Pre-LN

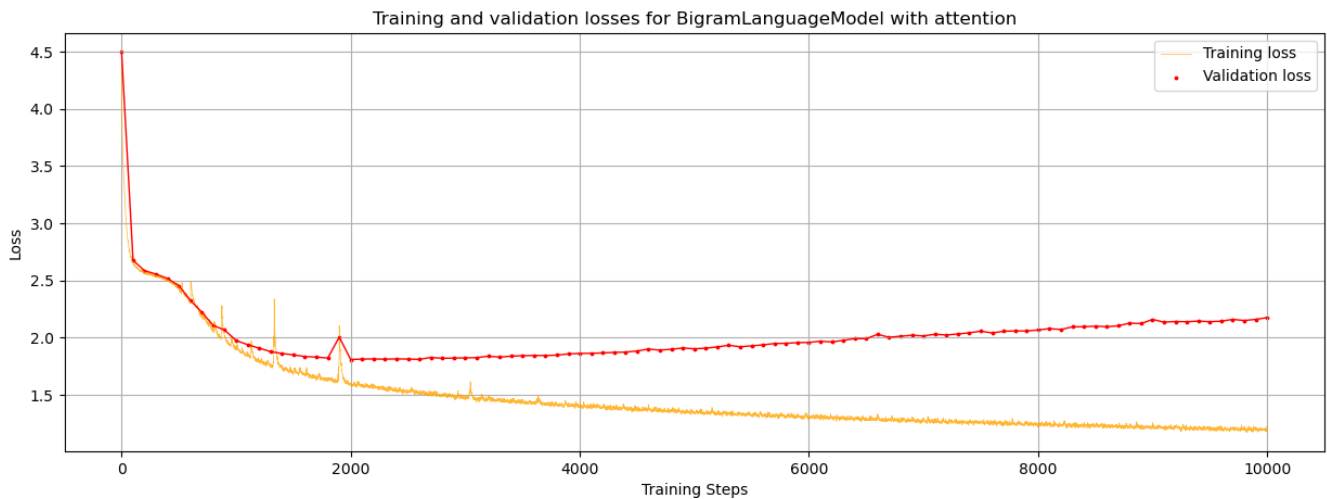


Рис. 3: Training и validation losses для BigramLanguageModel с нормализацией Post-LN (для 10000 итераций)

¹эксперимент для Post-LN-нормализации проведен на 10000 итерациях вместо 5000. Это влияет только на демонстрируемый ниже график обучения; показанные в таблице числа отражают состояние на момент 5000-ной итерации

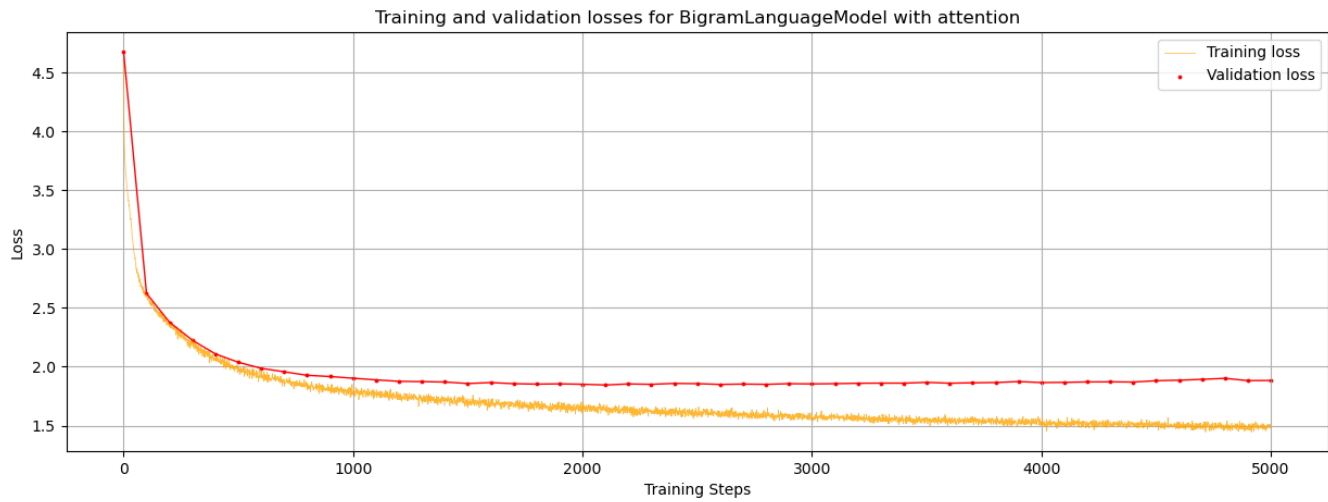


Рис. 4: Training и validation losses для BigramLanguageModel с нормализацией Sandwich-LN

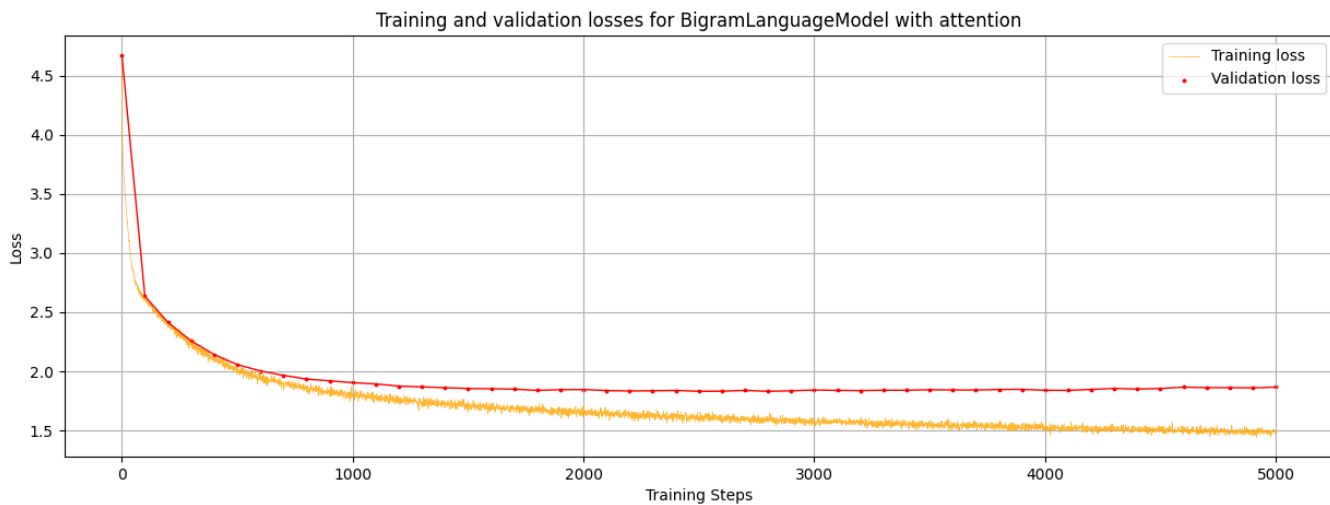


Рис. 5: Training и validation losses для BigramLanguageModel с нормализацией Pre-Post-LN

GroupNorm

Несмотря на рекомендации использовать `nn.LayerNorm` для трансформеров в общем случае, рассмотрим стабилизирующую обучение для малых батчей `nn.GroupNorm`. Введем дополнительный гиперпараметр:

- `num_groups = 8`,

и сравним 2 типа нормализации в расположении нормализации аргументов **Self-attention** и **Feed-forward** (Pre-GN и Pre-LN соответственно):

```

1 class Block(nn.Module):
2     """ Transformer block: communication followed by computation """
3
4     def __init__(self, n_embd, n_head):
5         <...>
6         self.gn1 = nn.GroupNorm(num_groups, n_embd)

```

```

7         self.gn2 = nn.GroupNorm(num_groups, n_embd)
8
9     def forward(self, x):
10         x = x + self.sa(
11             self.gn1(
12                 x.permute(0, 2, 1)      # (B,T,C) -> (B,C,T)
13             ).permute(0, 2, 1))          # (B,C,T) -> (B,T,C)
14         x = x + self.ffwd(
15             self.gn2(
16                 x.permute(0, 2, 1)      # (B,T,C) -> (B,C,T)
17             ).permute(0, 2, 1))          # (B,C,T) -> (B,T,C)
18         return x
19
20 class BigramLanguageModel(nn.Module):
21     """ Super simple bigram model """
22
23     def __init__(self):
24         <...>
25         self.gn_f = nn.GroupNorm(num_groups, n_embd)
26         <...>
27
28     def forward(self, idx, targets=None):
29         <...>
30         x = self.gn_f(
31             x.permute(0, 2, 1)          # (B,T,C) -> (B,C,T)
32         ).permute(0, 2, 1)              # (B,C,T) -> (B,T,C)

```

В таблице ниже отображены результаты обучения с групповой и послойной нормализацией. В каждом столбце жирным шрифтом выделен лучший показатель.

Тип и положение нормализации	Training loss на последнем шаге	Validation loss на последнем шаге	Лучший validation loss
Pre-LN (одинаковое расположение)	1.5294	1.8521	1.8308
Post-LN (лучший среди LN)	1.3426	1.9036	1.8098
Pre-GN	0.1434	0.1629	0.1576

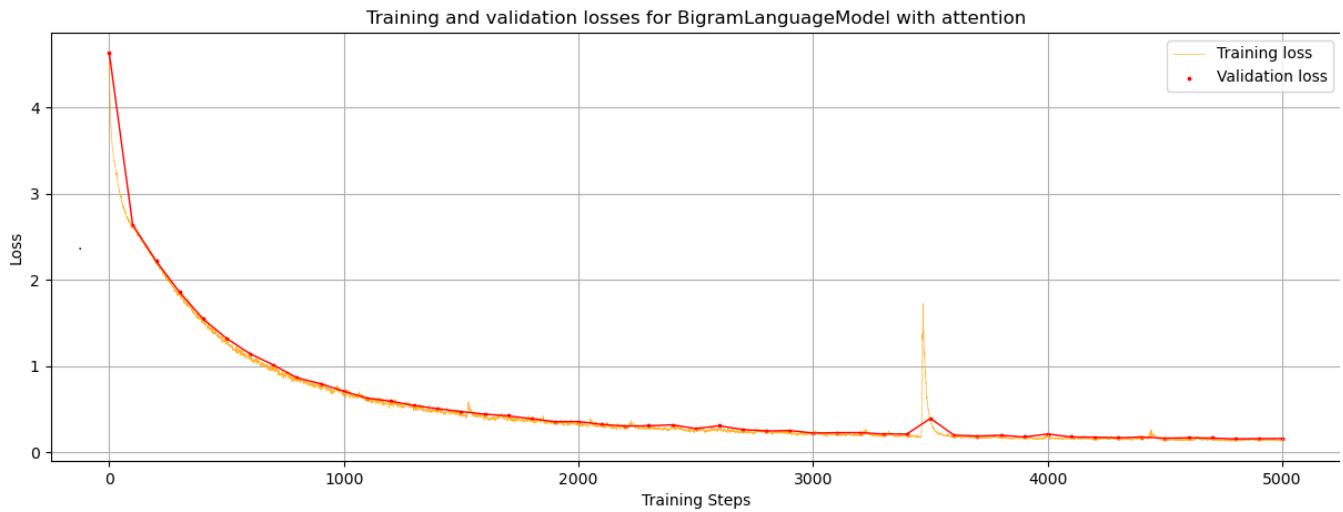


Рис. 6: Training и validation losses для BigramLanguageModel с нормализацией Pre-GN

Выводы

1. Отсутствие нормализации в сравнении с Layer и Group normalization:
 - (a) **Результативность обучения** с нормализацией Layer normalization при любом рассмотренном ее расположении выше, чем без нее;
 - (b) **Переобучение** без нормализации происходит медленнее;
2. Разные расположения Layer normalization в сравнении между собой:
 - (a) **Наибольшую результативность обучения** показывают модели с нормализацией Post-LN (показывает лучшие метрики на validation set);
 - (b) **Наибольшую скорость обучения** показывает модель с нормализацией Post-LN;
 - (c) **Наибольшую скорость приближения к нижнему предельному значению** показывает модель с нормализацией Sandwich-LN: ее тенденция training loss визуально наиболее выпукла, и loss быстрее остальных приближается к своему предельному значению (увы, по сравнению с остальными вариантами довольно высокому);
 - (d) **Наибольшую стабильность обучения** показывает модель с нормализацией Pre-LN: визуально разброс training loss на соответствующем графике меньше такового на остальных графиках;
 - (e) **Наименьшую стабильность обучения** показывает модель с нормализацией Post-LN: присутствуют выбросы (пики) в тенденциях графиках loss'ов;
 - (f) **Наиболее быстро переобучается** модель с нормализацией Post-LN.
3. Layer normalization в сравнении с Group normalization:
 - (a) **Наибольшую результативность обучения** показывает модель с нормализацией Pre-GN (показывает лучшие метрики на validation set);

- (b) **Наибольшую скорость обучения** показывает модель с нормализацией Pre-GN;
- (c) Оценка **стабильности обучения** варьируется: визуально разброс training loss на графике Pre-GN меньше такового на остальных графиках, но он также демонстрирует единичный существенный пик, отсутствие которых демонстрируется Pre-LN;
- (d) **Наиболее высокую обобщающую способность** демонстрирует модель с нормализацией Pre-GN (значения ее validation loss, отображенные на графике, практически не превышают значений training loss).

Вместе с тем, ручной просмотр примеров генерации показывает большую состоятельность моделей с именно послойной нормализацией в расположениях Pre- и Post-LN: сгенерированные ими тексты, пусть и лишены смысла, все же легко читаемы и содержат верно сгенерированные слова и словосочетания — в отличие от таковых для модели с групповой нормализацией.